



Dokumentace Maturitní Práce

Czech Jump Rope Register

Autor: Artur Smýkal

Vedoucí práce: Mgr. Markéta Fibigerová

Rok: 2026

Prohlášení

Prohlašuji, že jsem tuto práci vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), zejména se skutečností, že 1. IT Gymnázium, s. r. o. má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je 1. IT Gymnázium, s. r. o. oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Praze dne 1. dubna 2026

Prohlášení.....	1
1. Anotace.....	4
2. Úvod.....	5
3. Analýza problému a popis existujících řešení.....	6
3.1 Existující řešení na trhu.....	6
3.2 Možné technologie.....	6
4. Hlavní části softwarového projektu a jejich propojení.....	8
4.1 Frontend (Vue.js SPA).....	8
4.2 Backend (Firebase Cloud Functions).....	8
4.3 Databáze (Cloud Firestore).....	8
4.4 Komunikace mezi vrstvami.....	9
5. Use cases.....	10
5.1 Správa kolektivních členů (klubů).....	10
5.2 Správa evidovaných členů (jednotlivci).....	10
5.3 Import dat.....	10
5.4 Přehledy a reporty.....	10
5.5 Správa uživatelů.....	11
5.6 Validace adres.....	11
6. Datový model.....	12
6.1 Kolekce: collective_members.....	12
6.2 Kolekce: registered_members.....	13
7.3 Kolekce: authorized_users.....	14
6.4 Vztahy mezi entitami.....	15
7. Autentizace a autorizace.....	16
7.1 Autentizace.....	16
7.2 Autorizace.....	16
7.3 Technická implementace autorizace na backendu.....	16
8. Požadavky a odpovědi - přehled endpointů.....	18
9. Ošetření chybových stavů.....	19
9.1 HTTP stavové kódy API.....	19
9.2 Ošetření chyb na frontendu.....	19
10. Prostředí.....	20
11. Struktura zdrojového kódu.....	21
12. Použité technologie.....	22
12.1 Frontend.....	22
12.2 Backend (Cloud Functions).....	23
12.3 Platforma a infrastruktura.....	23
13. Závěr.....	25

14. Zdroje.....	26
Dokumentace použitých technologií.....	26
Dokumentace třetích stran a cloudové platformy.....	26
Ostatní technická dokumentace.....	27

1. Anotace

Tato práce se zabývá vývojem webové aplikace, která slouží jako evidenční systém pro základnu sportovní asociace Czech Jump Rope. Základna aktuálně používá Google Drive a Google Sheets pro evidenci členů. Nově vytvořený evidenční systém má sloužit jako náhrada aktuálního systému. Cílem bylo vytvořit aplikaci, která usnadní práci s daty a zajistí lepší bezpečnost a ochranu dat.

2. Úvod

Czech Jump Rope Register je webová aplikace určená ke správě členů základny sportovní asociace Czech Jump Rope. Aplikace umožňuje evidovat dva typy členů: kolektivní členy, tedy sportovní kluby a evidované členy, tedy jednotlivé sportovce, trenéry a rozhodčí. Evidovaný člen může mít příslušnost ke kolektivnímu členovi. Pokud evidovaný člen žádnou příslušnost nemá, je označen jako individuální člen.

Systém poskytuje nástroje pro hromadný import dat z CSV souborů, ruční zadávání a editaci záznamů prostřednictvím formulářů, export dat do různých formátů a správu přístupových práv uživatelů. Celá aplikace je navržena jako klient-server architektura s frontendovým rozhraním ve Vue.js a backendem postaveným na platformě Firebase.

3. Analýza problému a popis existujících řešení

Asociace Czech Jump Rope potřebovala nahradit dosavadní způsob evidence členů v tabulkových procesorech moderním webovým systémem. Stávající řešení trpěla absencí přístupových práv, nekonzistencí dat při sdílení souborů mezi uživateli, chybějící auditní stopou a nemožností generovat výstupy požadované Národní sportovní agenturou (NSA) pro dotační řízení. Cílem bylo zároveň vyvinout systém, který bude provozován za co nejmenší cenu, nebo ideálně zadarmo.

3.1 Existující řešení na trhu

Na trhu existují obecné registrační systémy, např. FAČR Registrace, IS pro sportovní svazy, avšak jsou buď finančně nákladné, vázané na konkrétní sportovní svaz, nebo nepodporují specifický formát NSA exportu vyžadovaný českou legislativou pro přidělování státních dotací sportovním svazům.

3.2 Možné technologie

Při výběru technologií byly zvažovány následující alternativy:

Oblast	Zvolená technologie	Alternativy
Frontend framework	Vue.js 3	React, Angular, Svelte
Backend / API	Firebase Cloud Functions (Node.js)	Express na vlastním VPS, Supabase Edge Functions
Databáze	Cloud Firestore (NoSQL)	PostgreSQL, MySQL, Firebase Realtime DB
Autentizace	Firebase Authentication	Auth0, vlastní JWT implementace
Hosting	Firebase Hosting	Vercel, Netlify, vlastní VPS
PDF generování	PDFKit (Node.js)	Puppeteer, jsPDF

Validace adres	RUIAN API (ČÚZK)	Smartform, Google Places API
----------------	------------------	------------------------------

Firebase platforma byla zvolena jako celek zejména z důvodu integrovaného ekosystému – autentizace, databáze, hosting i serverless funkce jsou spravovány z jednoho místa bez nutnosti konfigurace vlastní serverové infrastruktury. Vue.js 3 bylo zvoleno pro přehlednou komponentovou architekturu a reaktivní systém vhodný pro formulářově orientované aplikace. Firebase platforma účtuje ceny na základě výpočetního výkonu. V ceníku jsou určeny limity, pouze při překročení daných limitů začíná zákazník platit za výpočetní výkon. Rozsah aplikace je relativně malý a nemělo by dojít k překročení těchto limitů. Provoz aplikace tedy teoreticky bude probíhat zadarmo.

4. Hlavní části softwarového projektu a jejich propojení

Systém se skládá ze tří hlavních vrstev, které spolu komunikují přes definovaná rozhraní.

4.1 Frontend (Vue.js SPA)

Jednostránková aplikace (SPA) zajišťuje uživatelské rozhraní. Komunikuje s Firebase Authentication SDK přímo pro přihlášení a s backendovým REST API prostřednictvím HTTP požadavků s Bearer tokenem. S Firestore databází komunikuje frontend přímo pro čtení a zápis jednotlivých záznamů (přes Firestore SDK), zatímco hromadné operace (import, export, přesuny) deleguje na Cloud Functions API.

4.2 Backend (Firebase Cloud Functions)

Serverless REST API postavené na Express.js frameworku, nasazené v regionu europe-west3 (Frankfurt). Zajišťuje hromadné operace s daty, import dat, generování exportních souborů (CSV, PDF). Komunikuje s Firestore databází přes Firebase Admin SDK.

4.3 Databáze (Cloud Firestore)

NoSQL dokumentová databáze se dvěma hlavními kolekcemi: `collective_members` (kolektivní členové – kluby) a `registered_members` (evidovaní členové – jednotlivci). Kolekce je soubor dokumentů, každý dokument je JSON objekt.

Přístup k databázi je řízen bezpečnostními pravidly. Firestore Security Rules zabezpečují přímý přístup do databáze z frontendu. Firestore Security Rules je součástí softwaru, která na základě určených podmínek může povolit či odmítnout požadavek pro čtení nebo psaní dat. To zajišťuje pevnou strukturu dat, autentikaci a autorizaci.

Firebase Cloud Functions API má na starost složitější nebo náročnější požadavky. Komunikuje s databází přes Admin SDK, díky tomu zásahy do databáze z backendu nejsou kontrolovány pravidly Firestore Security Rules. Bezpečnost a zachování struktury dat ošetřuje kód na backendu.

4.4 Komunikace mezi vrstvami

Od	K	Protokol / SDK	Účel
Frontend	Firebase Auth	Firebase JS SDK	Google OAuth přihlášení, správa tokenů
Frontend	Cloud Firestore	Firebase JS SDK	CRUD operace s jednotlivými záznamy
Frontend	Cloud Functions API	HTTPS + Bearer token	Import, export, hromadné operace
Cloud Functions	Cloud Firestore	Firebase Admin SDK	Hromadné operace, privilegovaný zápis
Backend	RUIAN API	HTTPS REST	Validace českých adres

5. Use cases

5.1 Správa kolektivních členů (klubů)

- Přidání nového kolektivního člena přes formulář se „soft“ validací IČO, e-mailu, telefonu a adresy (RUIAN).
- Editace existujícího záznamu klubu.
- Ukončení členství klubu – nastaví datum zániku členství klubu a automaticky ukončí všechny jeho evidované členy.
- Smazání záznamu – pouze pro již ukončené kluby; kaskádově smaže všechny asociované evidované členy.
- Export aktivních klubů do CSV souboru.

5.2 Správa evidovaných členů (jednotlivci)

- Přidání nového evidovaného člena s vazbou na klub nebo jako individuální člen.
- Editace záznamu včetně rolí (sportovec, trenér, rozhodčí), dat a adresy.
- Přesun členů mezi kluby nebo na individuální členství.
- Export členů konkrétního klubu do CSV.
- Export pro NSA (Národní sportovní agenturu) – specializovaný formát pro státní dotace.

5.3 Import dat

- Hromadný import kolektivních nebo evidovaných členů z CSV souboru (oddělovač středník).
- Náhled dat před importem s validačními upozorněními na úrovni jednotlivých řádků.
- Podpora aktualizace existujících záznamů při přítomnosti sloupce id.

5.4 Přehledy a reporty

- Generování přehledového PDF dokumentu se statistikami členské základny (počty aktivních klubů, sportovců, trenérů, rozhodčích) a tabulkou s rozpisem podle klubů.

5.5 Správa uživatelů

- Přidání uživatele do whitelistu s přiřazenou rolí (admin, editor, čtenář).
- Změna role existujícího uživatele.
- Odebrání uživatele.

5.6 Validace adres

- Integrace s RUIAN API (Registr územní identifikace, adres a nemovitostí, ČÚZK) pro ověření platnosti české adresy v reálném čase.
- Výsledky validace jsou cachovány na straně frontendu, aby se zamezilo překročení limitu 1 000 dotazů za hodinu.

6. Datový model

Databáze využívá Cloud Firestore NoSQL dokumentové úložiště. Níže jsou popsány tři kolekce a jejich datový model.

6.1 Kolekce: `collective_members`

Pole	Typ	Popis
id (doc ID)	string	Automaticky generovaný identifikátor UID
name	string null	Název klubu
company_id	string null	IČO (identifikační číslo organizace)
address.street	string null	Ulice
address.house_number	string null	Číslo popisné / orientační
address.zip_code	string null	PSČ
address.township	string null	Obec
address.country	string null	Stát
contact_person.first_name	string null	Jméno kontaktní osoby
contact_person.last_name	string null	Příjmení kontaktní osoby
contact_person.email	string null	E-mail kontaktní osoby
contact_person.phone_number	string null	Telefon kontaktní osoby

membership_origin_date	Timestamp null	Datum vzniku členství
membership_extinction_date	Timestamp null	Datum zániku členství (null = aktivní)
createdAt	Timestamp	Datum a čas vytvoření záznamu
createdBy	string	E-mail uživatele, který záznam vytvořil
modifiedAt	Timestamp	Datum a čas poslední úpravy
modifiedBy	string	E-mail uživatele, který naposledy upravil záznam

6.2 Kolekce: registered_members

Pole	Typ	Popis
id (doc ID)	string	Automaticky generovaný identifikátor UID
first_name	string null	Jméno
last_name	string null	Příjmení
birth_number	string null	Rodné číslo (10 číslic)
sex	string null	Pohlaví: M, Z nebo Ž
date_of_birth	Timestamp null	Datum narození (musí být v minulosti)
address.*	string null	Adresa (stejná struktura jako u klubů)

nationality_code	string null	Kód státní příslušnosti (ISO 3166-1 alpha-3, např. CZE)
membership_origin_date	Timestamp null	Datum vzniku členství
membership_extinction_date	Timestamp null	Datum zániku členství (null = aktivní)
medical_examination_validity	Timestamp null	Platnost lékařské prohlídky
competitions_last_12_months	number null	Počet soutěží za posledních 12 měsíců
athlete	boolean null	Je sportovec
referee	boolean null	Je rozhodčí
coach	boolean null	Je trenér
collective_member_ref	string null	ID nadřazeného klubu (FK na collective_members)
createdAt / createdBy	Timestamp / string	Auditní pole - vytvoření
modifiedAt / modifiedBy	Timestamp / string	Auditní pole - úprava

7.3 Kolekce: authorized_users

Pole	Typ	Popis
id (doc ID)	string	E-mailová adresa uživatele (malá písmena)

role	string	Role uživatele: admin editor viewer
------	--------	---

6.4 Vztahy mezi entitami

Pole `collective_member_ref` v kolekci `registered_members` obsahuje ID dokumentu z kolekce `collective_members` a reprezentuje vztah N:1 (více evidovaných členů může být členem jednoho klubu). Pole je nullable – evidovaný člen bez vazby na klub je považován za individuálního člena.

Referenční integrita je vynucována jak na úrovni Firestore Security Rules (frontend přístup), tak v logice Cloud Functions (API přístup). Při ukončení nebo smazání klubu jsou všechny jeho evidované členy kaskádově ovlivněny.

7. Autentizace a autorizace

7.1 Autentizace

Autentizace je implementována prostřednictvím Firebase Authentication s poskytovatelem Google OAuth 2.0. Uživatel se přihlásí kliknutím na tlačítko 'Přihlásit se přes Google', čímž se otevře přihlašovací popup okno Google. Po úspěšném přihlášení Firebase vygeneruje JWT ID Token s platností 1 hodiny, který se automaticky obnovuje.

Samotné přihlášení k Firebase Auth ale nestačí – systém udržuje samostatný whitelist v kolekci `authorized_users`. Při každém přihlášení (nebo obnovení stránky) je zkontrolováno, zda e-mailová adresa přihlášeného uživatele existuje v kolekci `authorized_users`. Pokud ne, uživatel je považován za nepovoleného a je mu odepřen přístup k aplikaci.

7.2 Autorizace

Role	Popis oprávnění
viewer (čtenář)	Může pouze prohlížet záznamy členů. Nemá přístup k formulářům ani importu/exportu.
editor	Vše co viewer + může přidávat a editovat záznamy, importovat a exportovat data, provádět přesuny a ukončení členství.
admin	Vše co editor, dále může spravovat seznam povolených uživatelů (přidat, změnit roli, odebrat).

7.3 Technická implementace autorizace na backendu

Každý HTTP požadavek na Cloud Functions API musí obsahovat hlavičku `Authorization: Bearer <ID_TOKEN>`. Middleware `validateRole` provede:

- Extrakci tokenu z hlavičky `Authorization`.
- Ověření tokenu pomocí Firebase Admin SDK (`admin.auth().verifyIdToken(token)`) – ověří podpis a platnost tokenu.

- Dotaz do Firestore kolekce `authorized_users` na základě e-mailu z dekodovaného tokenu.
- Kontrolu, zda role uživatele patří mezi role povolené pro daný endpoint.
- Při úspěchu připojí informace o uživateli (e-mail, role) do `req` objektu a předá řízení handleru.

Na úrovni Firestore Security Rules je přístup řízen funkcemi `isAdmin()`, `isEditor()`, `isViewer()` a `isEditorOrAdmin()`, které čtou roli z `authorized_users` při každém přístupu k databázi z frontendu.

8. Požadavky a odpovědi - přehled endpointů

Metoda	Cesta	Role	Popis
POST	/api/collectives/import	admin, editor	Hromadný import klubů z CSV dat
GET	/api/collectives/export	admin, editor	Export aktivních klubů jako CSV
POST	/api/collectives/terminate	admin, editor	Ukončení členství klubu a všech příslušných sportovců
DELETE	/api/collectives/delete	admin, editor	Smazání ukončeného klubu a všech příslušných sportovců
POST	/api/registered/import	admin, editor	Hromadný import evidovaných členů
GET	/api/registered/export/:collectiveId	admin, editor	Export členů konkrétního klubu
GET	/api/registered/export-nsa	admin, editor	Export ve formátu NSA
POST	/api/registered/transfer	admin, editor	Přesun členů mezi kluby
GET	/api/reports/overview	admin, editor	Generování přehledu základny ve formátu PDF
GET	/api/validate-address	admin, editor, čtenář	Ověření adresy přes RUIAN

9. Ošetření chybových stavů

9.1 HTTP stavové kódy API

Kód	Název	Kdy se vrátí
200 OK	Úspěch	Operace proběhla úspěšně
400 Bad Request	Chybný požadavek	Chybějící nebo neplatná data (např. neplatný formát data, chybějící povinné pole, pokus smazat aktivní klub)
401 Unauthorized	Neautorizováno	Chybějící nebo expirovaný Bearer token
403 Forbidden	Zakázáno	Uživatel není na whitelistu nebo nemá dostatečnou roli pro daný endpoint
404 Not Found	Nenalezeno	Dokument s daným ID v databázi neexistuje
500 Internal Server Error	Serverová chyba	Neošetřená výjimka, selhání databázové operace

9.2 Ošetření chyb na frontendu

- Chyby z API jsou zachyceny v servisních vrstvách (services) a předány do Pinia errorStore.
- ErrorStore zobrazí chybovou zprávu jako globální alert banner na vrchu stránky.
- Validací chyby ve formulářích jsou zobrazeny u příslušného pole.
- Import zobrazí validační varování na úrovni řádků před odesláním dat.

10. Prostředí

Prostředí	Frontend URL	Backend	Databáze
Produkce	https://czech-jump-rope-register-e8dea.web.app	Cloud Functions (europe-west3)	Cloud Firestore (europe-west3)
Alternativní produkce	https://czech-jump-rope-register-e8dea.firebaseio.com	Cloud Functions (europe-west3)	Cloud Firestore (europe-west3)
Lokální vývoj	http://localhost:5173 (Vite dev server)	Functions Emulator :5001	Firestore Emulator :8080, Auth Emulator :9099

Lokální vývojové prostředí využívá Firebase Local Emulator Suite. Aplikace automaticky detekuje vývojový režim (`import.meta.env.DEV` v prostředí Vite) a připojí se k lokálním emulátorům místo produkčních Firebase služeb. Produkční URL API (`/api`) je v produkci přesměrována přes Firebase Hosting rewrite pravidla na Cloud Function api.

Citlivé konfigurační hodnoty (API klíče, Firebase credentials) jsou spravovány přes `.env` soubory: `.env.local` pro lokální vývoj a `.env.production` pro produkční build. Soubor `.env.local` je zahrnut v `.gitignore` a není verzován.

11. Struktura zdrojového kódu

Zdrojový kód projektu je verzován v systému Git. Struktura repozitáře:

```
czech-jump-rope-register/  
├── frontend/                                # Vue.js 3 SPA  
│   ├── src/  
│   │   ├── views/                          # Stránky aplikace  
│   │   ├── components/                     # Sdílené komponenty  
│   │   ├── services/                       # API a business logika  
│   │   ├── stores/                         # Pinia stores (stav)  
│   │   ├── router/                         # Vue Router konfigurace  
│   │   └── firebase.js                     # Inicializace Firebase SDK  
│   ├── .env.local                          # Lokální env proměnné (není verzováno)  
│   ├── .env.production                     # Produkční env proměnné  
│   └── vite.config.js                      # Vite konfigurace  
├── backend/  
│   ├── functions/                          # Firebase Cloud Functions  
│   │   ├── src/  
│   │   │   ├── routes/                    # Express route handlers  
│   │   │   └── middleware/                 # Auth middleware  
│   │   ├── index.js                       # Vstupní bod - Express app + CORS  
│   │   └── package.json  
├── firestore.rules                          # Firestore Security Rules  
├── firestore.indexes.json                  # Firestore indexy  
├── firebase.json                           # Firebase konfigurace nasazení  
└── .firebaserc                            # Firebase projekt reference
```

12. Použité technologie

12.1 Frontend

Technologie / Knihovna	Verze	Účel
Vue.js	^3.5.18	Frontend framework – reaktivní UI
Vue Router	^4.5.1	Klientské směrování (SPA routing)
Pinia	^3.0.4	Správa globálního stavu aplikace
Vite	^7.0.6	Build nástroj, dev server, HMR
Firebase JS SDK	^12.1.0	Auth a Firestore klient
PapaParse	^5.5.3	Parsování CSV souborů na klientovi
rodnecislo	^1.8.0	Validace českého rodného čísla
validator	^13.15.26	Obecné validace (e-mail, URL aj.)
vite-plugin-vue-devtools	^8.0.0	Vue DevTools integrace (pouze dev)
npm-run-all2	^8.0.4	Paralelní spouštění npm skriptů

12.2 Backend (Cloud Functions)

Technologie / Knihovna	Verze	Účel
Node.js	22	Runtime prostředí
Firebase Functions SDK	^6.0.1	Definice Cloud Functions (2. generace)
Firebase Admin SDK	^12.6.0	Serverový přístup k Firebase službám
Express.js	^5.1.0	HTTP framework pro REST API
CORS	^2.8.6	Middleware pro Cross-Origin Resource Sharing
PDFKit	^0.18.0	Programatické generování PDF dokumentů

12.3 Platforma a infrastruktura

Služba	Poskytovatel	Účel
Firebase Hosting	Google Firebase	Statický hosting SPA frontendu
Cloud Functions (2. gen.)	Google Firebase / Cloud Run	Serverless backend API
Cloud Firestore	Google Firebase	NoSQL dokumentová databáze
Firebase Authentication	Google Firebase	Správa uživatelů, Google OAuth

Artifact Registry	Google Cloud	Úložiště Docker image pro Cloud Functions
Cloud Build	Google Cloud	CI/CD build pro Cloud Functions
RUIAN API	ČÚZK (státní správa ČR)	Validace českých adres

13. Závěr

Cílem této práce bylo vyvinout webovou aplikaci, která nahradí stávající způsob evidence členů sportovní asociace Czech Jump Rope. Výsledkem je funkční systém Czech Jump Rope Register, který pokrývá všechny identifikované požadavky – správu kolektivních i evidovaných členů, hromadný import a export dat, generování PDF přehledů, export ve formátu NSA pro účely státních dotací a správu přístupových práv uživatelů.

Aplikace je postavena na klient-server architektuře s frontendovým rozhraním ve Vue.js 3 a backendovými službami na platformě Firebase. Tato volba se ukázala jako vhodná zejména díky integrovanému ekosystému Firebase, který umožnil nasadit autentizaci, databázi, serverless API i statický hosting bez nutnosti spravovat vlastní serverovou infrastrukturu. Provoz aplikace by měl při předpokládaném rozsahu dat zůstat v bezplatném pásmu platformy.

Systém řeší klíčové nedostatky původního řešení: zavádí granulární řízení přístupových práv na základě rolí, zajišťuje konzistenci dat prostřednictvím Firestore Security Rules a validační logiky na backendu a poskytuje auditní stopu ke každému záznamu.

Během vývoje bylo největší výzvou správné nastavení bezpečnostních pravidel Firestore tak, aby byla data přístupná pouze oprávněným uživatelům, a zároveň aby nedocházelo k zbytečnému omezení běžných operací. Další výzvou byla integrace s RUIAN API včetně cachování výsledků s ohledem na hodinový limit dotazů.

Do budoucna by bylo možné systém rozšířit například o notifikace při blížícím se vypršení lékařské prohlídky, pokročilé filtrování a vyhledávání v přehledech nebo o mobilní optimalizaci rozhraní. Aplikace je nicméně v současném stavu připravena k produkčnímu nasazení a plně nahrazuje dosavadní řešení v Google Sheets.

14. Zdroje

Dokumentace použitých technologií

- Vue.js 3 – <https://vuejs.org/guide/introduction.html>
- Vue Router 4 – <https://router.vuejs.org/>
- Pinia – <https://pinia.vuejs.org/>
- Vite – <https://vite.dev/guide/>
- Firebase Documentation – <https://firebase.google.com/docs>
 - Firebase Authentication – <https://firebase.google.com/docs/auth>
 - Cloud Firestore – <https://firebase.google.com/docs/firestore>
 - Firebase Hosting – <https://firebase.google.com/docs/hosting>
 - Cloud Functions (2. gen.) – <https://firebase.google.com/docs/functions>
 - Firestore Security Rules – <https://firebase.google.com/docs/firestore/security/get-started>
- Firebase Local Emulator Suite – <https://firebase.google.com/docs/emulator-suite>
- Firebase CLI Reference – <https://firebase.google.com/docs/cli>
- Express.js 5 – <https://expressjs.com/>
- PDFKit – <https://pdfkit.org/>
- PapaParse – <https://www.papaparse.com/docs>
- rodneCislo (npm) – <https://www.npmjs.com/package/rodneCislo>
- validator.js – <https://github.com/validatorjs/validator.js>

Dokumentace třetích stran a cloudové platformy

- Google Cloud IAM – <https://cloud.google.com/iam/docs>
- Google Cloud Functions Troubleshooting – <https://cloud.google.com/functions/docs/troubleshooting>
- Google Cloud Artifact Registry – <https://cloud.google.com/artifact-registry/docs>
- RUIAN API (ČÚZK) – <https://ruian.fnx.io/>

- Národní sportovní agentura – datový formát NSA –
<https://www.agenturasport.cz/>

Ostatní technická dokumentace

- Git – <https://git-scm.com/doc>
- Node.js 22 Release Notes –
<https://nodejs.org/en/blog/release/v22.0.0>
- CORS specifikace –
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- ISO 3166-1 alpha-3 (kódy státní příslušnosti) –
<https://www.iso.org/iso-3166-country-codes.html>
- CSV injection – OWASP –
https://owasp.org/www-community/attacks/CSV_Injection